

Python versions

A Python version is composed of a Python interpreter (i.e. the python executable), the standard library, and other supporting files.

Managed and system Python installations

Since it is common for a system to have an existing Python installation, uv supports discovering Python versions. However, uv also supports installing Python versions itself. To distinguish between these two types of Python installations, uv refers to Python versions it installs as *managed* Python installations and all other Python installations as *system* Python installations.

!!! note

uv does not distinguish between Python versions installed by the operating system vs those installed and managed by other tools. For example, if a Python installation is managed with ``pyenv``, it would still be considered a `_system_` Python version in uv.

Requesting a version

A specific Python version can be requested with the `--python` flag in most uv commands. For example, when creating a virtual environment:

```
$ uv venv --python 3.11.6
```

uv will ensure that Python 3.11.6 is available — downloading and installing it if necessary — then create the virtual environment with it.

The following Python version request formats are supported:

- `<version>` (e.g., 3, 3.12, 3.12.3)
- `<version-specifier>` (e.g., `>=3.12`, `<3.13`)
- `<version><short-variant>` (e.g., 3.13t, 3.12.0d)
- `<version>+<variant>` (e.g., 3.13+freethreaded, 3.12.0+debug, 3.14+gil)
- `<implementation>` (e.g., cpython or cp)
- `<implementation>@<version>` (e.g., cpython@3.12)
- `<implementation><version>` (e.g., cpython3.12 or cp312)
- `<implementation><version-specifier>` (e.g., cpython>=3.12,<3.13)
- `<implementation>-<version>-<os>-<arch>-<libc>` (e.g., cpython-3.12.3-macos-aarch64-none)

Additionally, a specific system Python interpreter can be requested with:

- `<executable-path>` (e.g., /opt/homebrew/bin/python3)
- `<executable-name>` (e.g., mypython3)
- `<install-dir>` (e.g., /some/environment/)

By default, uv will automatically download Python versions if they cannot be found on the system. This behavior can be disabled with the `python-downloads` option.

Python version files

The `.python-version` file can be used to create a default Python version request. uv searches for a `.python-version` file in the working directory and each of its parents. If none is found, uv will check the user-level configuration directory. Any of the request formats described above can be used, though use of a version number is recommended for interoperability with other tools.

A `.python-version` file can be created in the current directory with the `uv python pin` command.

A global `.python-version` file can be created in the user configuration directory with the `uv python pin --global` command.

Discovery of `.python-version` files can be disabled with `--no-config`.

`uv` will not search for `.python-version` files beyond project or workspace boundaries (except the user configuration directory).

Installing a Python version

`uv` bundles a list of downloadable CPython and PyPy distributions for macOS, Linux, and Windows.

!!! tip

By default, Python versions are automatically downloaded as needed without using ``uv python install``.

To install a Python version at a specific version:

```
$ uv python install 3.12.3
```

To install the latest patch version:

```
$ uv python install 3.12
```

To install a version that satisfies constraints:

```
$ uv python install '>=3.8,<3.10'
```

To install multiple versions:

```
$ uv python install 3.9 3.10 3.11
```

To install a specific implementation:

```
$ uv python install pypy
```

All the Python version request formats are supported except those that are used for requesting local interpreters such as a file path.

By default `uv python install` will verify that a managed Python version is installed or install the latest version. If a `.python-version` file is present, `uv` will install the Python version listed in the file. A project that requires multiple Python versions may define a `.python-versions` file. If present, `uv` will install all the Python versions listed in the file.

!!! important

The available Python versions are frozen for each `uv` release. To install new Python versions, you may need upgrade `uv`.

See the storage documentation for details about where installed Python versions are stored.

Installing Python executables

`uv` installs Python executables into your `PATH` by default, e.g., on Unix `uv python install 3.12` will install a Python executable into `~/.local/bin`, e.g., as `python3.12`. See the storage documentation for more details about the target directory.

!!! tip

If `~/.local/bin` is not in your ``PATH``, you can add it with ``uv python update-shell``.

To install `python` and `python3` executables, include the experimental `--default` option:

```
$ uv python install 3.12 --default
```

When installing Python executables, uv will only overwrite an existing executable if it is managed by uv — e.g., if `~/.local/bin/python3.12` exists already uv will not overwrite it without the `--force` flag.

uv will update executables that it manages. However, it will prefer the latest patch version of each Python minor version by default. For example:

```
$ uv python install 3.12.7 # Adds `python3.12` to `~/.local/bin`  
$ uv python install 3.12.6 # Does not update `python3.12`  
$ uv python install 3.12.8 # Updates `python3.12` to point to 3.12.8
```

Upgrading Python versions

!!! important

Upgrades are only supported for uv-managed Python versions.

Upgrades are not currently supported for PyPy, GraalPy, and Pyodide.

uv allows transparently upgrading Python versions to the latest patch release, e.g., 3.13.4 to 3.13.5.

uv does not allow transparently upgrading across minor Python versions, e.g., 3.12 to 3.13, because changing minor versions can affect dependency resolution.

uv-managed Python versions can be upgraded to the latest supported patch release with the `python upgrade` command:

To upgrade a Python version to the latest supported patch release:

```
$ uv python upgrade 3.12
```

To upgrade all installed Python versions:

```
$ uv python upgrade
```

After an upgrade, uv will prefer the new version, but will retain the existing version as it may still be used by virtual environments.

Virtual environments using the Python version will be automatically upgraded to the new patch version.

If a virtual environment was created with an explicitly requested patch version, e.g., `uv venv -p 3.10.8`, it will not be transparently upgraded to a new version.

Minor version directories

Automatic upgrades for virtual environments are implemented using a directory with the Python minor version, e.g.:

```
~/.local/share/uv/python/cpython-3.12-macos-aarch64-none
```

which is a symbolic link (on Unix) or junction (on Windows) pointing to a specific patch version:

```
$ readlink ~/.local/share/uv/python/cpython-3.12-macos-aarch64-none  
~/.local/share/uv/python/cpython-3.12.11-macos-aarch64-none
```

If this link is resolved by another tool, e.g., by canonicalizing the Python interpreter path, and used to create a virtual environment, it will not be automatically upgraded.

Project Python versions

uv will respect Python requirements defined in `requires-python` in the `pyproject.toml` file during project command invocations. The first Python version that is compatible with the requirement will be used, unless a version is otherwise requested, e.g., via a `.python-version` file or the `--python` flag.

Viewing available Python versions

To list installed and available Python versions:

```
$ uv python list
```

To filter the Python versions, provide a request, e.g., to show all Python 3.13 interpreters:

```
$ uv python list 3.13
```

Or, to show all PyPy interpreters:

```
$ uv python list pypy
```

By default, downloads for other platforms and old patch versions are hidden.

To view all versions:

```
$ uv python list --all-versions
```

To view Python versions for other platforms:

```
$ uv python list --all-platforms
```

To exclude downloads and only show installed Python versions:

```
$ uv python list --only-installed
```

See the `uv python list` reference for more details.

Finding a Python executable

To find a Python executable, use the `uv python find` command:

```
$ uv python find
```

By default, this will display the path to the first available Python executable. See the discovery rules for details about how executables are discovered.

This interface also supports many request formats, e.g., to find a Python executable that has a version of 3.11 or newer:

```
$ uv python find '>=3.11'
```

By default, `uv python find` will include Python versions from virtual environments. If a `.venv` directory is found in the working directory or any of the parent directories or the `VIRTUAL_ENV` environment variable is set, it will take precedence over any Python executables on the `PATH`.

To ignore virtual environments, use the `--system` flag:

```
$ uv python find --system
```

Discovery of Python versions

When searching for a Python version, the following locations are checked:

- Managed Python installations in the `UV_PYTHON_INSTALL_DIR`.
- A Python interpreter on the `PATH` as `python`, `python3`, or `python3.x` on macOS and Linux, or `python.exe` on Windows.

- On Windows, the Python interpreters in the Windows registry and Microsoft Store Python interpreters (see `py --list-paths`) that match the requested version.

In some cases, uv allows using a Python version from a virtual environment. In this case, the virtual environment's interpreter will be checked for compatibility with the request before searching for an installation as described above. See the [pip-compatible virtual environment discovery documentation](#) for details.

When performing discovery, non-executable files will be ignored. Each discovered executable is queried for metadata to ensure it meets the requested Python version. If the query fails, the executable will be skipped. If the executable satisfies the request, it is used without inspecting additional executables.

When searching for a managed Python version, uv will prefer newer versions first. When searching for a system Python version, uv will use the first compatible version — not the newest version.

If a Python version cannot be found on the system, uv will check for a compatible managed Python version download.

Python pre-releases

Python pre-releases will not be selected by default. Python pre-releases will be used if there is no other available installation matching the request. For example, if only a pre-release version is available it will be used but otherwise a stable release version will be used. Similarly, if the path to a pre-release Python executable is provided then no other Python version matches the request and the pre-release version will be used.

If a pre-release Python version is available and matches the request, uv will not download a stable Python version instead.

Free-threaded Python

uv supports discovering and installing free-threaded Python variants in CPython 3.13+.

For Python 3.13, free-threaded Python versions will not be selected by default. Free-threaded Python versions will only be selected when explicitly requested, e.g., with `3.13t` or `3.13+freethreaded`.

For Python 3.14+, uv will allow use of free-threaded Python 3.14+ interpreters without explicit selection. The GIL-enabled build of Python will still be preferred, e.g., when performing an installation with `uv python install 3.14`. However, e.g., if a free-threaded interpreter comes before a GIL-enabled build on the `PATH`, it will be used.

If both free-threaded and GIL-enabled Python versions are available on the system, and want to require the use of the GIL-enabled variant in a project, you can use the `+gil` variant specifier.

Debug Python variants

uv supports discovering and installing debug builds of Python, i.e., with debug assertions enabled.

!!! important

Debug builds of Python are slower and are not appropriate for general use.

Debug builds will be used if there is no other available installation matching the request. For example, if only a debug version is available it will be used but otherwise a stable release version will be used. Similarly, if the path to a debug Python executable is provided then no other Python version matches the request and the debug version will be used.

Debug builds of Python can be explicitly requested with, e.g., `3.13d` or `3.13+debug`.

!!! note

CPython versions installed by uv usually have debug symbols stripped to reduce the distribution size. These debug builds do not have debug symbols stripped, which can be useful when debugging Python processes with a C-level debugger.

Disabling automatic Python downloads

By default, uv will automatically download Python versions when needed.

The `python-downloads` option can be used to disable this behavior. By default, it is set to `automatic`; set to `manual` to only allow Python downloads during `uv python install`.

!!! tip

The ``python-downloads`` setting can be set in a [persistent configuration file](./configuration-files.md) to change the default behavior, or the ``--no-python-downloads`` flag can be passed to any uv command.

Requiring or disabling managed Python versions

By default, uv will attempt to use Python versions found on the system and only download managed Python versions when necessary. To ignore system Python versions, and only use managed Python versions, use the `--managed-python` flag:

```
$ uv python list --managed-python
```

Similarly, to ignore managed Python versions and only use system Python versions, use the `--no-managed-python` flag:

```
$ uv python list --no-managed-python
```

To change uv's default behavior in a configuration file, use the `python-preference` setting.

Adjusting Python version preferences

The `python-preference` setting determines whether to prefer using Python installations that are already present on the system, or those that are downloaded and installed by uv.

By default, the `python-preference` is set to `managed` which prefers managed Python installations over system Python installations. However, system Python installations are still preferred over downloading a managed Python version.

The following alternative options are available:

- `only-managed`: Only use managed Python installations; never use system Python installations. Equivalent to `--managed-python`.
- `system`: Prefer system Python installations over managed Python installations.
- `only-system`: Only use system Python installations; never use managed Python installations. Equivalent to `--no-managed-python`.

!!! note

Automatic Python version downloads can be [disabled](#disabling-automatic-python-downloads) without changing the preference.

Python implementation support

uv supports the CPython, PyPy, Pyodide, and GraalPy Python implementations. If a Python implementation is not supported, uv will fail to discover its interpreter.

The implementations may be requested with either the long or short name:

- CPython: cpython, cp
- PyPy: pypy, pp
- GraalPy: graalpy, gp
- Pyodide: pyodide

Implementation name requests are not case-sensitive.

See the Python version request documentation for more details on the supported formats.

Managed Python distributions

uv supports downloading and installing CPython, PyPy, and Pyodide distributions.

CPython distributions

As Python does not publish official distributable CPython binaries, uv instead uses pre-built distributions from the Astral python-build-standalone project. python-build-standalone is also used in many other Python projects, like Mise and bazelbuild/rules_python.

The uv Python distributions are self-contained, highly-portable, and performant. While Python can be built from source, as in tools like pyenv, doing so requires preinstalled system dependencies, and creating optimized, performant builds (e.g., with PGO and LTO enabled) is very slow.

These distributions have some behavior quirks, generally as a consequence of portability; see the python-build-standalone quirks documentation for details.

PyPy distributions

!!! note

PyPy versions lag behind CPython and currently only supports Python versions up to 3.11

PyPy distributions are provided by the PyPy project.

Pyodide distributions

Pyodide distributions are provided by the Pyodide project.

Pyodide is a port of CPython for the WebAssembly / Emscripten platform.

Transparent x86_64 emulation on aarch64

Both macOS and Windows support running x86_64 binaries on aarch64 through transparent emulation. This is called Rosetta 2 or Windows on ARM (WoA) emulation. It's possible to use x86_64 uv on aarch64, and also possible to use an x86_64 Python interpreter on aarch64. Either uv binary can use either Python interpreter, but a Python interpreter needs packages for its architecture, either all x86_64 or all aarch64.

Registration in the Windows registry

On Windows, installation of managed Python versions will register them with the Windows registry as defined by PEP 514.

After installation, the Python versions can be selected with the py launcher, e.g.:

```
$ uv python install 3.13.1  
$ py -V:Astral/CPython3.13.1
```

On uninstall, uv will remove the registry entry for the target version as well as any broken registry entries.